UniConnect

Software Design and Software Requirements Specification Report

Software Project Lab II (SWE 4404)

Team Number	:	4
Project Name	:	UniConnect
Team Members	:	Maliha Tasnim Khan (230042127) Sayma Tasnim (230042139) Ayman Binta Altaf Nondiny (230042141) Saika Sarara (230042159)
Submission Date	:	July 27, 2026

Contents

List of Figures

Chapter 1

Project Overview

1.1 Introduction

UniConnect is a university-exclusive networking and collaboration platform designed to connect students, alumni, club operators, and system administrators within a centralized digital ecosystem. It combines verified account access, profile-based discovery, connection management, project teammate finding, peer mentorship, one-to-one chat, club announcements, a career board, global search, notifications, and administrative moderation.

1.2 Problem Statement

University communication is commonly fragmented across social media, messaging groups, email, and informal personal networks. As a result, students may struggle to find collaborators with the required skills, juniors may not know which seniors can provide mentorship, alumni career opportunities may be missed, club announcements may be buried, and administrators may lack an official moderation channel. UniConnect addresses these problems through a verified, role-aware platform.

1.3 Objectives

- establish a trusted university-only digital community;
- help students discover collaborators by skills and network proximity;
- provide structured mentorship slots for scalable peer support;
- enable real-time communication between eligible users;
- centralize club announcements and role management;
- provide an alumni-driven career opportunity board;
- support platform search, notifications, and accountable administration.

1.4 Target Users

- **Students**: networking, projects, mentorship, announcements, and career discovery.
- **Alumni**: professional profiles and job or internship publishing.
- **Club Operators**: club administration and official announcement publishing.
- **System Administrators**: user management, moderation, club governance, and audit.

1.5 Technology and Development Approach

The proposed solution uses Flutter for the client application, Java Spring Boot for backend services, PostgreSQL for persistent storage, WebSocket communication for real-time chat, and JWT with BCrypt for authentication. The development approach is modular object-oriented design with SOLID principles and an Agile lifecycle.

1.6 Project Repository

https://github.com/aymannondiny/SPL-II_UniConnect

Chapter 2

Class Diagram Design

2.1 Reading Order

The diagrams are presented from the broadest domain model to the most specialized modules:

1. High-Level Domain Model
2. Authentication, Authorization and Profile Management
3. Club and Announcement Management
4. Connection and Networking
5. Project Teammate Finder
6. Peer Mentorship
7. One-to-One Chat
8. Career Board
9. Administration and Notification Management

This order begins with shared identity entities, then introduces relationship-dependent collaboration features, and ends with cross-cutting governance.

2.2 High-Level Class Diagram

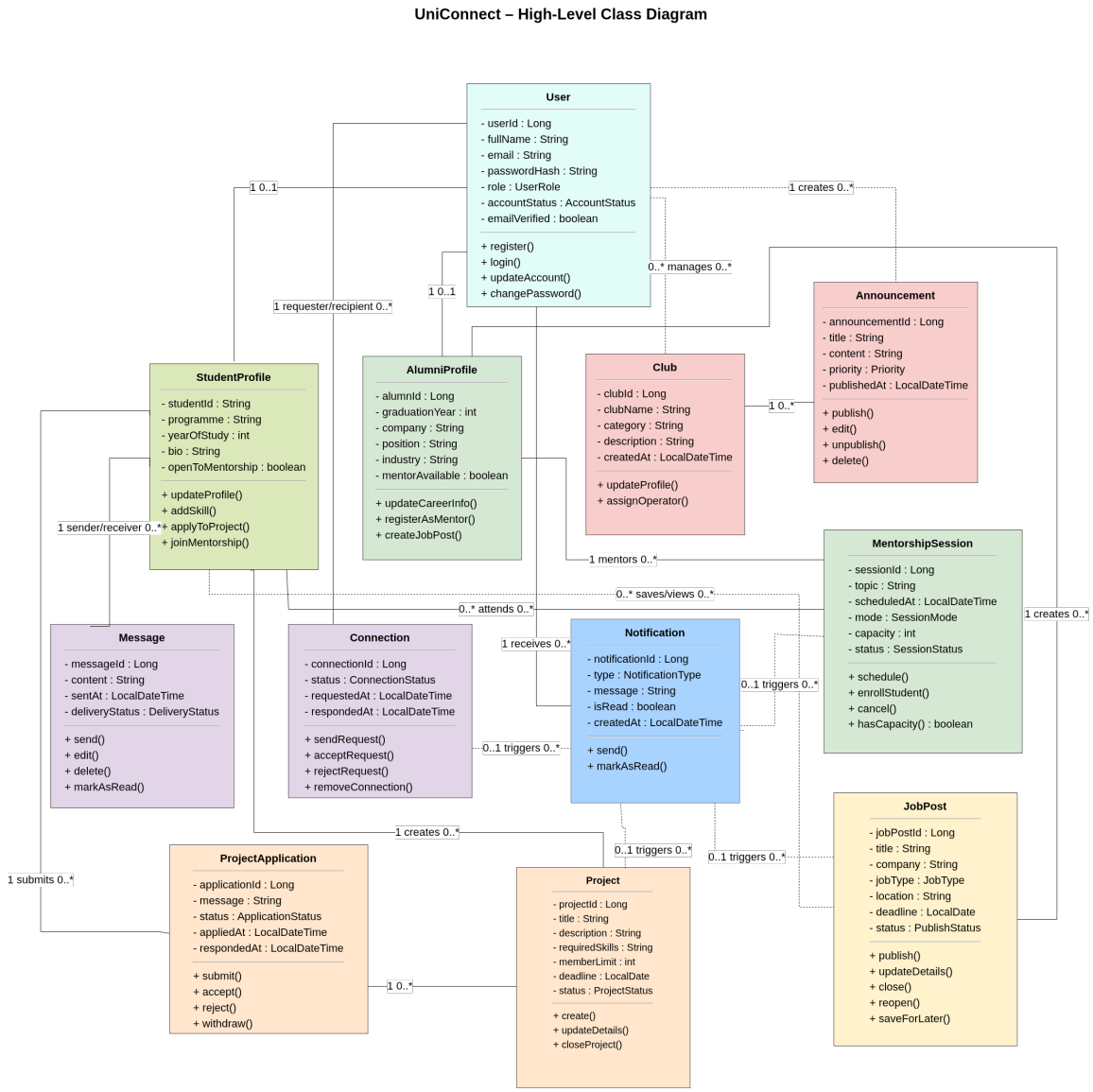

Figure 2.1: High-Level Class Diagram

Explanation. The high-level diagram provides the domain overview of UniConnect. It identifies the central User abstraction and the major feature entities: StudentProfile, AlumniProfile, Club, Announcement, Connection, Message, Project, ProjectApplication, MentorshipSession, JobPost, and Notification. It is intentionally compact and should be read before the feature-specific diagrams because it establishes the vocabulary and the principal relationships used throughout the design.

2.3 Authentication, Authorization and Profile Management

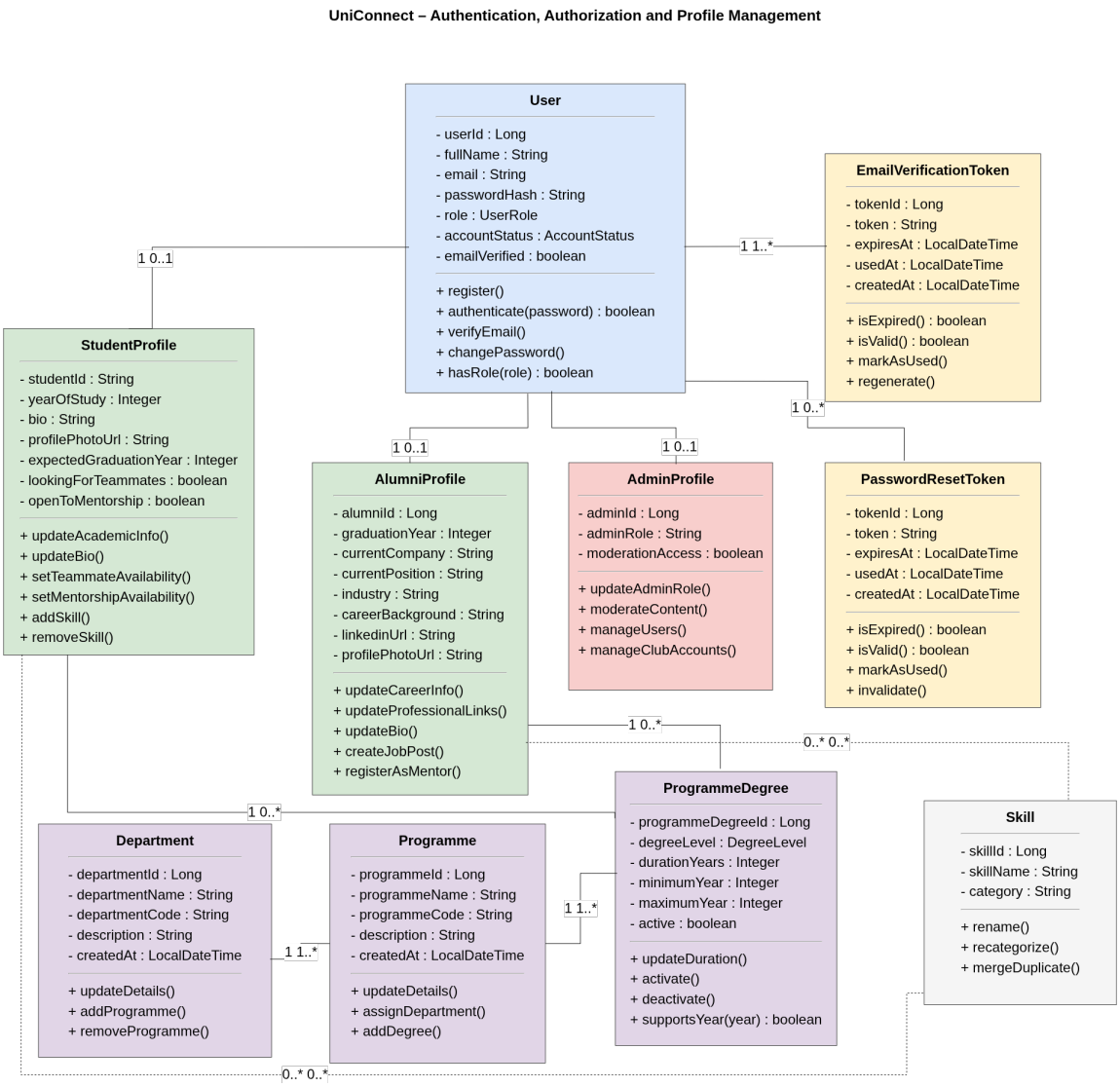

Figure 2.2: Authentication, Authorization and Profile Management

Explanation. This diagram expands identity management. User stores shared account data, while StudentProfile, AlumniProfile, and AdminProfile hold role-specific information. EmailVerificationToken and PasswordResetToken support secure account activation and recovery. Department, Programme, ProgrammeDegree, and Skill provide reusable academic reference data. The design keeps authentication data separate from profile data and supports role-based authorization.

2.4 Club and Announcement Management

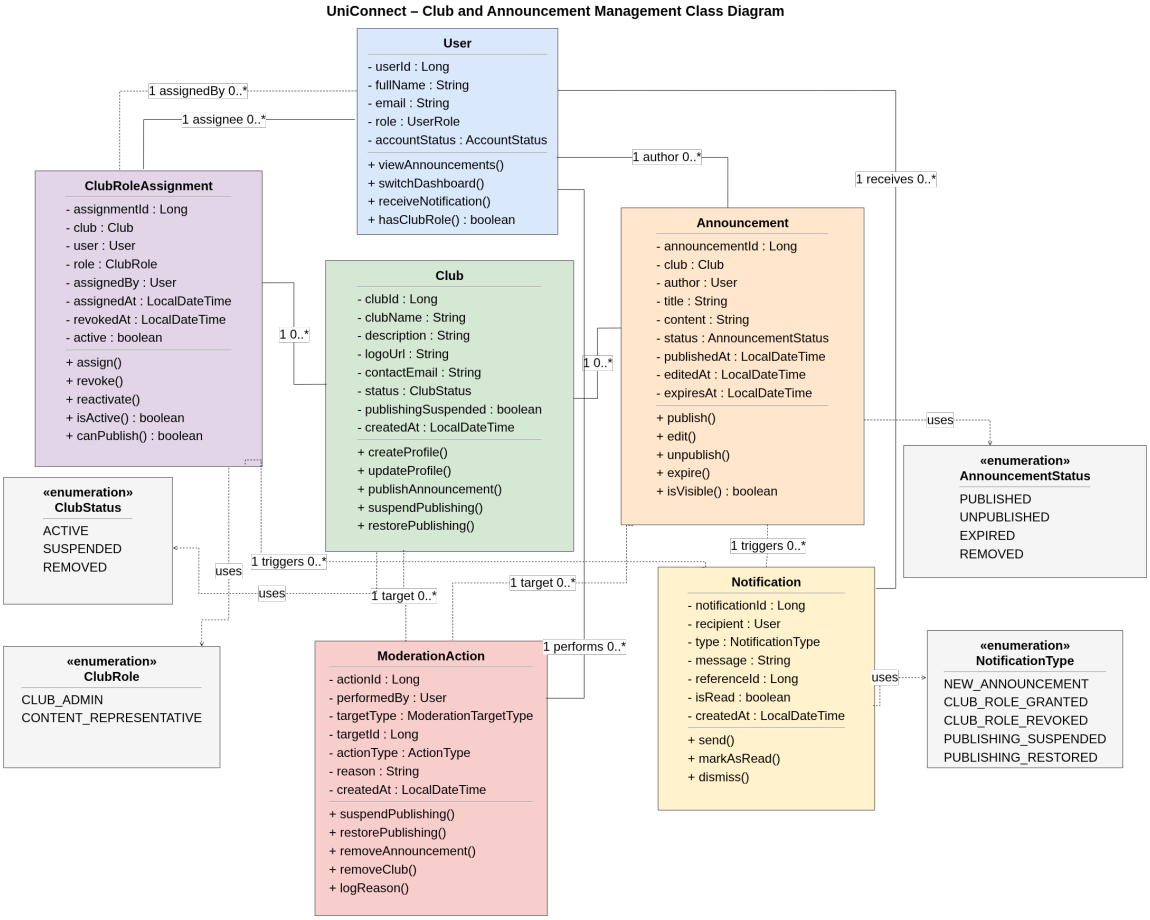

Figure 2.3: Club and Announcement Management

Explanation. This diagram models clubs as organizational entities rather than ordinary user profiles. ClubRoleAssignment connects a User to a Club with a role such as Club Admin or Content Representative. Announcement belongs to a Club and is authored by an authorized operator. Notification informs users about announcements and role changes, while ModerationAction records administrative intervention such as publishing suspension.

2.5 Connection and Networking

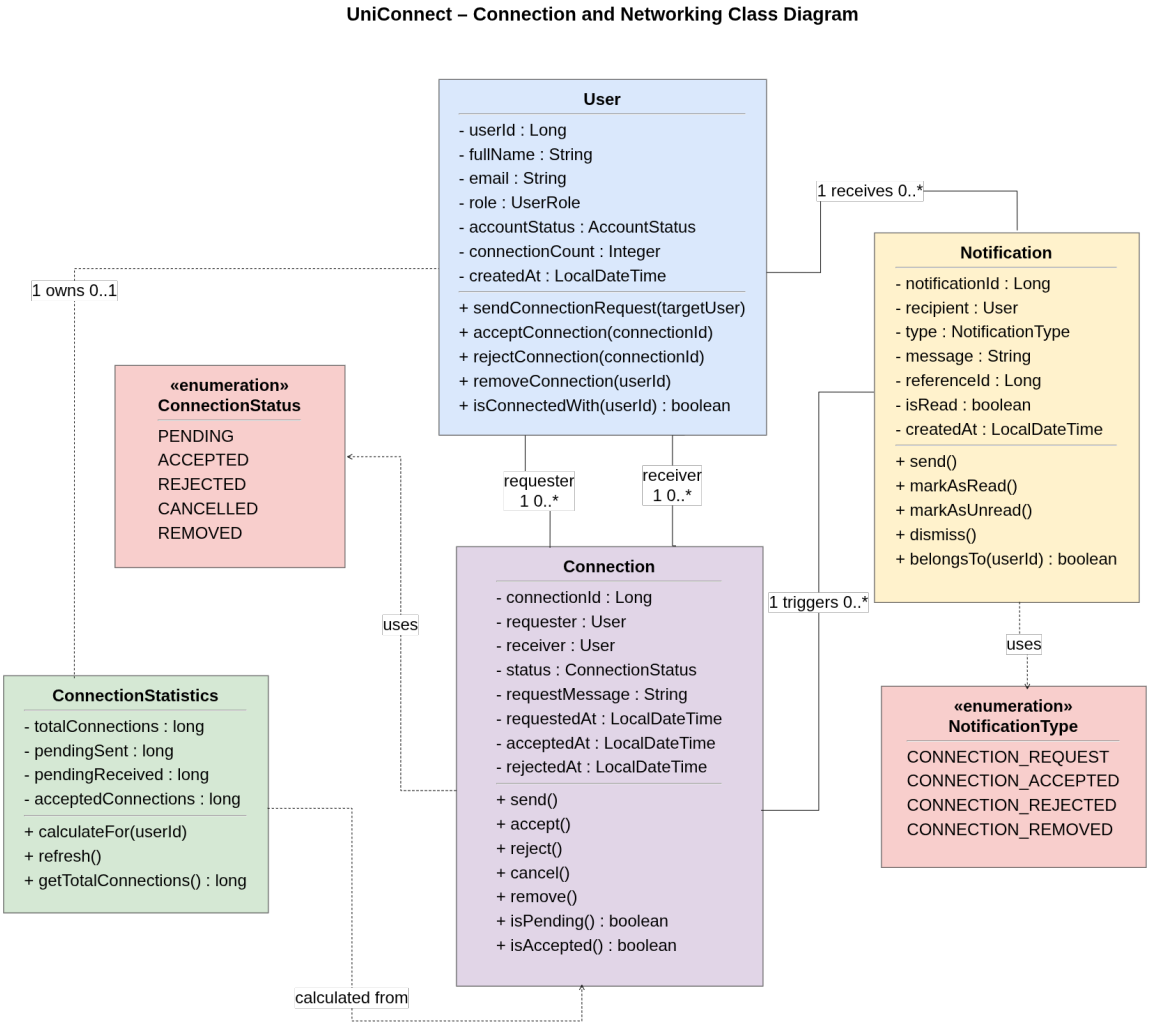

Figure 2.4: Connection and Networking

Explanation. This diagram defines the social graph. A Connection links a requester and receiver and moves through pending, accepted, rejected, cancelled, or removed states. Accepted connections contribute to ConnectionStatistics and enable downstream communication. Notification records connection requests and responses. The design centralizes relationship validation so other modules can rely on a consistent first-degree network.

2.6 Project Teammate Finder

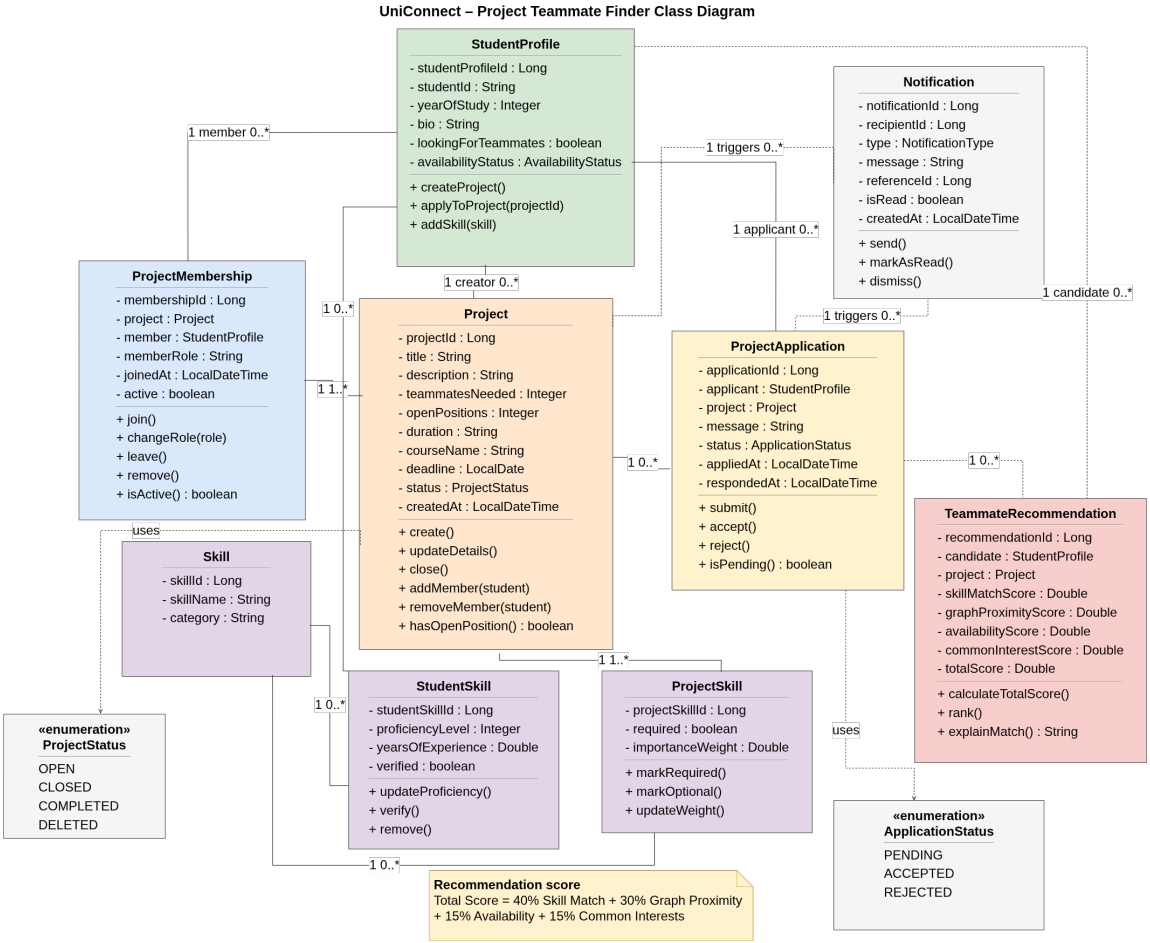

Figure 2.5: Project Teammate Finder

Explanation. This diagram represents project recruitment and algorithmic teammate discovery. Project stores recruitment information, ProjectApplication captures candidate requests, and ProjectMembership records the accepted team. StudentSkill and ProjectSkill normalize skill data, while TeammateRecommendation stores the calculated match components. The recommendation combines skill match, graph proximity, availability, and common interests.

2.7 Peer Mentorship

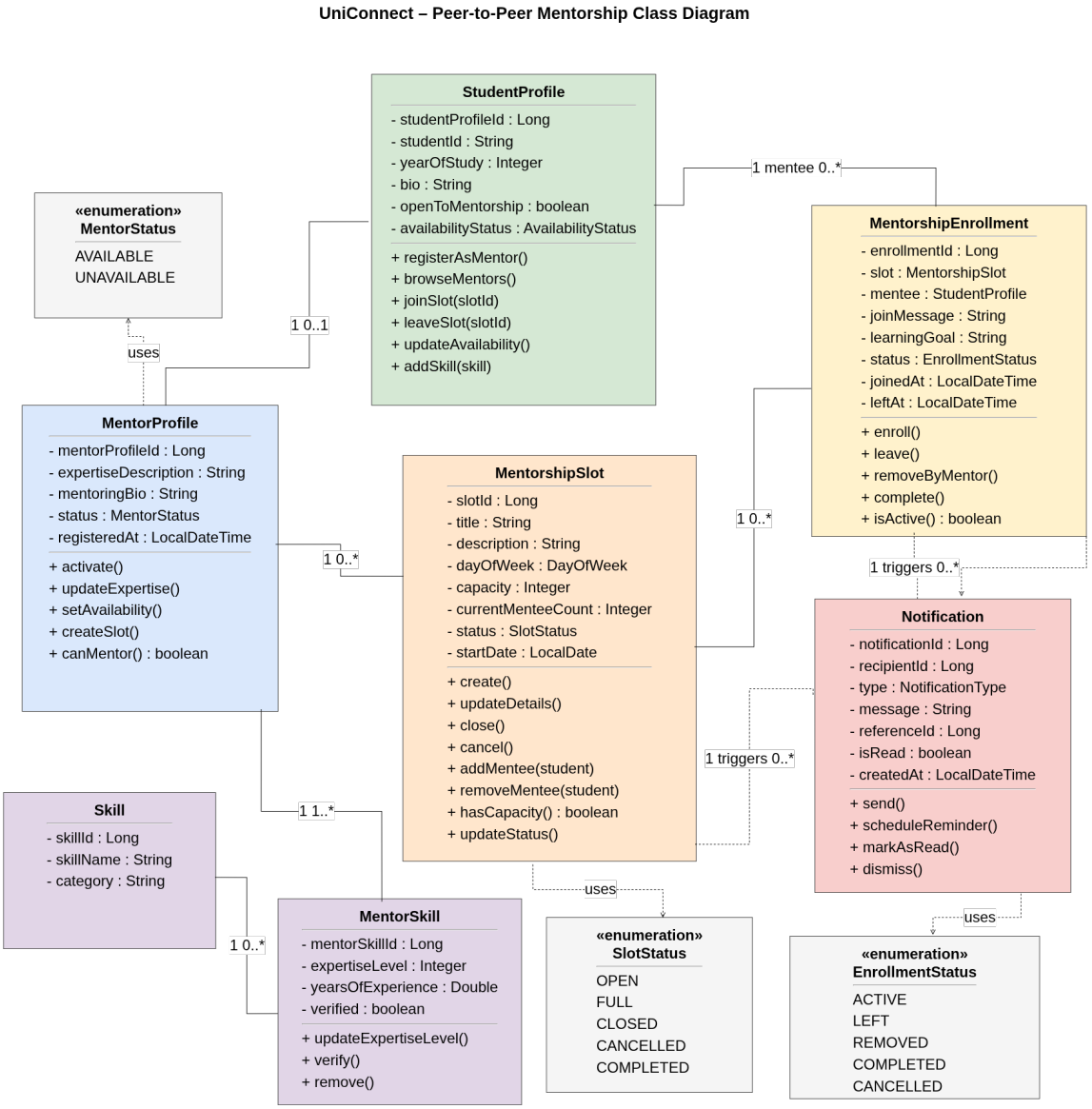

Figure 2.6: Peer Mentorship

Explanation. This diagram models mentors, scheduled mentorship slots, and mentee enrollment. MentorProfile extends a student's mentorship capability, MentorshipSlot controls schedule and capacity, and MentorshipEnrollment represents the many-to-many relationship between students and slots. MentorSkill links mentors with expertise areas, while Notification supports enrollment confirmations, schedule changes, and reminders.

2.8 One-to-One Chat

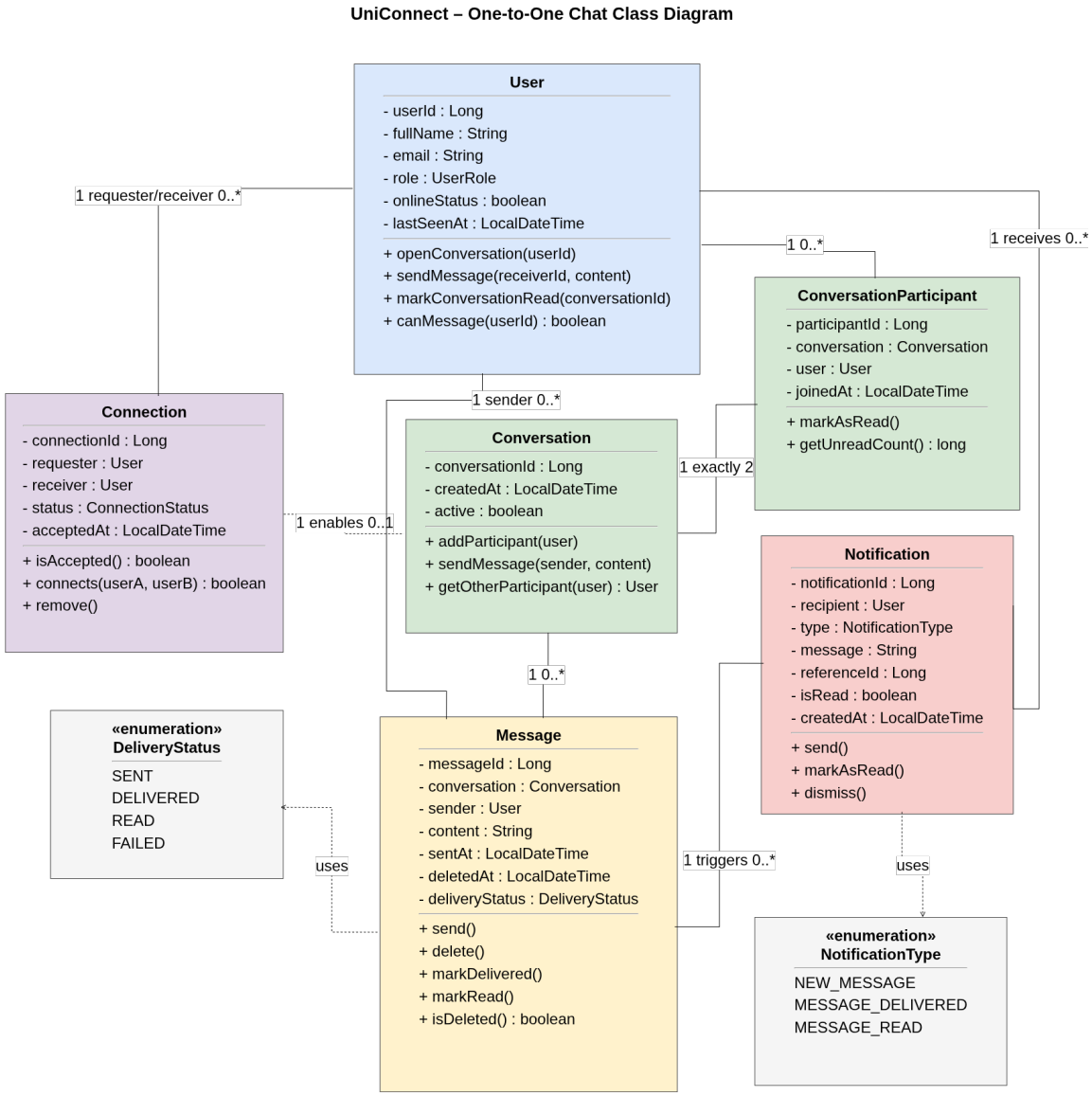

Figure 2.7: One-to-One Chat

Explanation. This diagram models real-time private communication. Conversation groups two participants, ConversationParticipant stores per-user state such as unread count, and Message stores durable chat history and delivery status. Connection is consulted before a conversation can be used, enforcing the relationship restriction. Notification supports background message alerts and read or delivery events.

2.9 Career Board

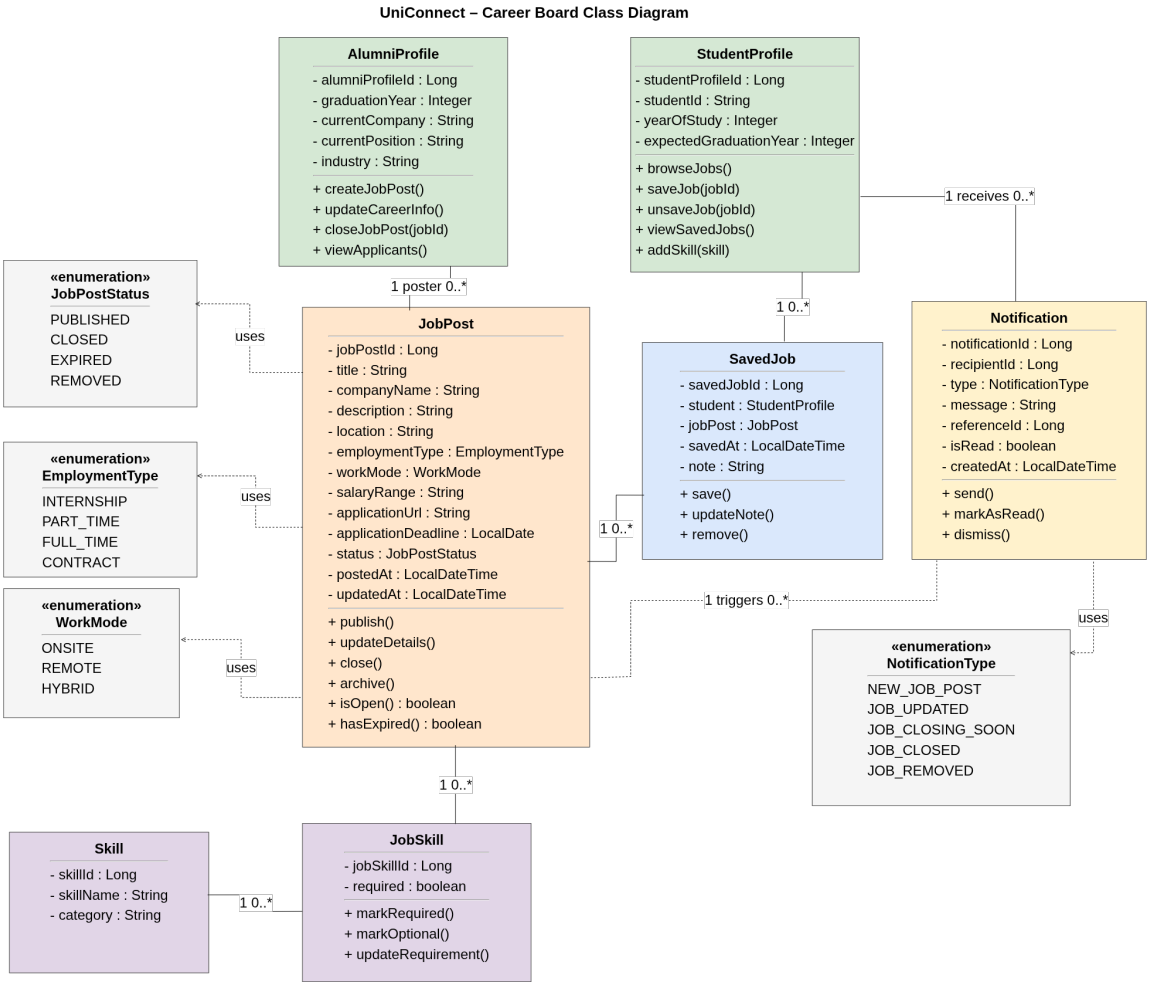

Figure 2.8: Career Board

Explanation. This diagram models alumni-posted career opportunities. AlumniProfile owns JobPost records, JobSkill describes required or optional competencies, and SavedJob allows students to maintain a shortlist. EmploymentType, WorkMode, and JobPostStatus constrain valid states. Notification can be used for post updates, approaching deadlines, closure, or removal.

2.10 Administration and Notification Management

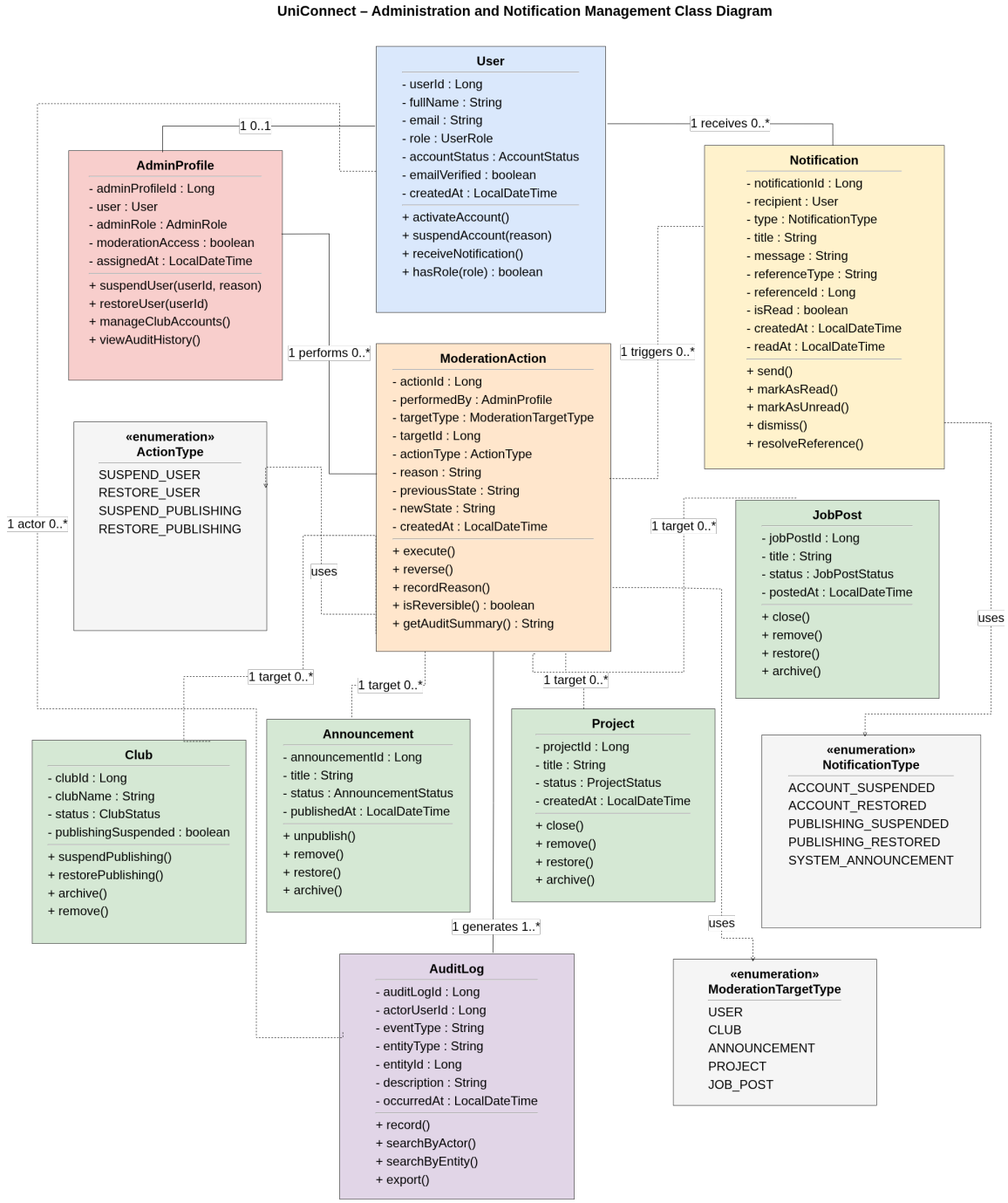

Figure 2.9: Administration and Notification Management

Explanation. This final feature diagram covers cross-cutting governance. AdminProfile performs ModerationAction against users, clubs, announcements, projects, and jobs. AuditLog preserves administrative traceability. Notification delivers account, publishing, and system-level changes. It is placed last because it supervises and integrates with nearly every earlier module.

Chapter 3

System Design Architecture

3.1 Architecture Style

UniConnect follows a **three-tier architecture** with **MVC organization inside the presentation tier**. This structure separates interface concerns, application rules, and persistent data access.

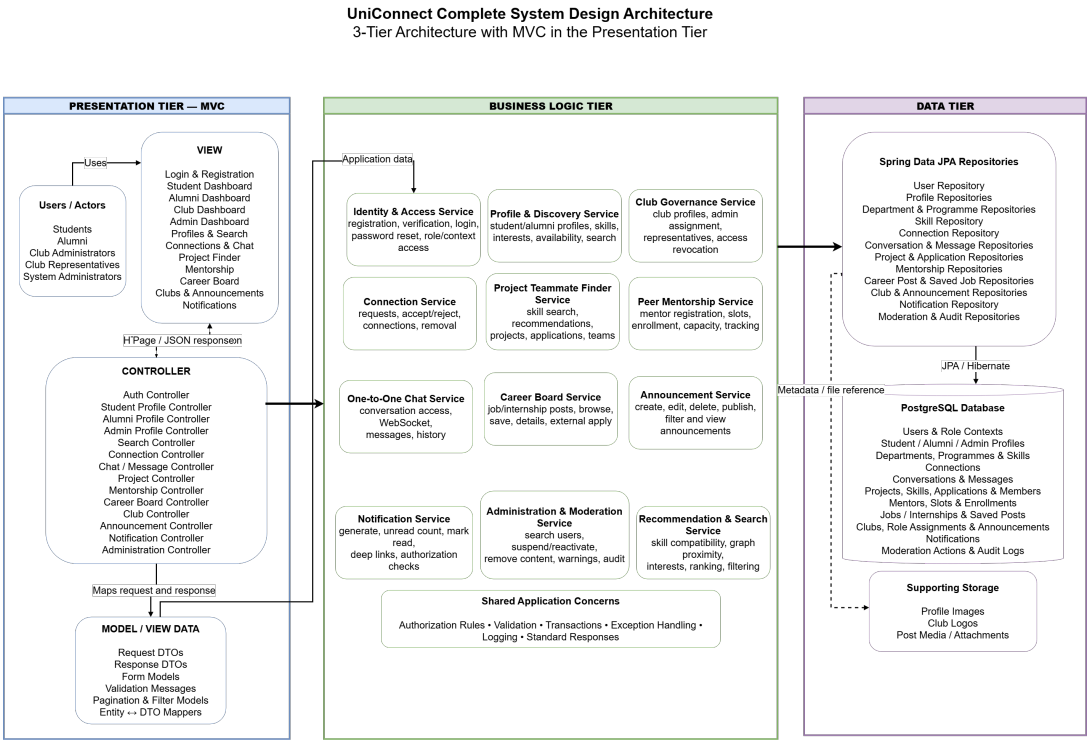

Figure 3.1: UniConnect Complete 3-Tier MVC Vertical Architecture

Explanation. The presentation tier contains Flutter views, controllers, request and response models, validation messages, and entity–DTO mapping. Controllers expose feature-oriented endpoints for authentication, profiles, clubs, search, connections, chat, projects, mentorship, career opportunities, announcements, notifications, and administration.

The business logic tier contains dedicated services for each module and shared concerns such as authorization, validation, transactions, logging, and exception handling. The data tier uses Spring Data JPA repositories backed by PostgreSQL, with supporting storage for images and attachments.

The vertical organization keeps each feature traceable from the user interface through controller, service, repository, and database.

3.2 Request Flow

1. A user performs an action in a Flutter view.
2. The associated controller validates the request model and authentication context.
3. A feature service applies authorization and business rules.
4. A repository reads or writes persistent entities.
5. The service returns a response DTO.
6. The controller returns a standardized response to the view.

3.3 Architectural Benefits

The design improves maintainability because user-interface changes do not directly modify database code. Services can be tested independently, repositories isolate persistence details, and shared authorization rules can be consistently enforced. The vertical feature organization also makes it easier for team members to develop different modules without excessive coupling.

Chapter 4

Detailed Software Requirements Specification

4.1 SRS Structure

Each requirement below is transcribed from the requirement collection workbook. Every entry contains the functional requirement, linked user story, acceptance criteria, priority, and current implementation status. User stories are presented continuously without forcing a separate page for each entry. The workbook contains 72 functional requirement identifiers and 81 user stories because several functional requirements are decomposed into multiple user stories.

4.2 Priority Definitions

- **High**: essential functionality required for the core system or a major operational workflow.
- **Medium**: important supporting functionality that improves completeness and usability.
- **Low**: functionality that may be implemented after higher-priority workflows.

4.3 Authentication and Authorization

4.3.1 US_1: Create a Verified University Account

Functional Requirement ID: FR_1 **User Story ID:** US_1 **Priority:** High **Status:** Done

Functional Requirement

The system shall allow an eligible university member to create an account using an approved university email address.

User Story

As an eligible University Member, I want to create an account using my university-approved email address, so that I can join the UniConnect platform.

Acceptance Criteria

- Full name, university email, password, and user type are collected.
- Only approved university email domains are accepted.
- Duplicate emails are rejected.
- Password must contain at least 8 characters.
- Account is created with Unverified status.

4.3.2 US_2.1: Confirm University Affiliation

Functional Requirement ID: FR_2 **User Story ID:** US_2.1 **Priority:** High **Status:** Done

Functional Requirement

The system shall allow a registered university member to verify their account through a verification link sent to their university email address.

User Story

As a newly registered member, I want to verify my university email, so that the platform can trust my institutional affiliation and activate my account.

Acceptance Criteria

- Verification email is sent after registration.
- Link expires after 24 hours.
- Valid link changes status to Verified.
- Invalid or expired link shows an error.
- Used token cannot be reused.

4.3.3 US_2.2: Request a New Verification Link

Functional Requirement ID: FR_2 **User Story ID:** US_2.2 **Priority:** Low **Status:** To Do

Functional Requirement

The system shall allow a registered university member to verify their account through a verification link sent to their university email address.

User Story

As a newly registered member whose verification link has expired, I want to request another link so that I can complete activation.

Acceptance Criteria

- The member can request a new verification email using the registered university email address.
- The previous expired token remains invalid.
- The newly issued link has a new validity period.
- The response does not expose whether an unrelated email address is registered.

4.3.4 US_3: Access My UniConnect Account

Functional Requirement ID: FR_3 **User Story ID:** US_3 **Priority:** High **Status:** Done

Functional Requirement

The system shall authenticate a verified university member using their registered email address and password.

User Story

As a verified university member, I want to log in using my registered email and password, so that I can securely access my account.

Acceptance Criteria

- Login form accepts email and password.
- Valid credentials create a session.
- Unverified accounts are blocked.
- Invalid credentials show an error.
- User is redirected to the correct dashboard.

4.3.5 US_4: Recover Forgotten Account Access

Functional Requirement ID: FR_4 **User Story ID:** US_4 **Priority:** High **Status:** Done

Functional Requirement

The system shall allow a registered university member to reset a forgotten password through a time-limited email link.

User Story

As a registered university member, I want to reset my forgotten password, so that I can recover access to my account without administration assistance.

Acceptance Criteria

- Reset link can be requested using a registered email.
- A time-limited link is sent.
- New password must satisfy password rules.
- Expired links are rejected.
- New password works for login.
- Old password is deleted.

4.3.6 US_5.1: View Role-Relevant Functions

Functional Requirement ID: FR_5 **User Story ID:** US_5.1 **Priority:** High **Status:** In Progress

Functional Requirement

The system shall restrict platform and club functions according to the authenticated user's platform role, active context, and club-role assignments.

User Story

As a registered member, I want to see only the platform functions relevant for my role, so that I can avoid inaccessible actions.

Acceptance Criteria

- Student-context functions are shown when Student context is active.
- Alumni-context functions are shown when Alumni access applies.
- Club-management functions are shown only when an active Club context is selected.
- Functions unavailable in the current context are hidden or disabled.
- Changing context refreshes the visible navigation and available actions.

4.3.7 US_5.2: Protect Assigned Club Resources

Functional Requirement ID: FR_5 **User Story ID:** US_5.2 **Priority:** High **Status:** In Progress

Functional Requirement

The system shall restrict platform and club functions according to the authenticated user's platform role, active context, and club-role assignments.

User Story

As a Club Admin, I want club-management permissions to apply only to my assigned club so that another club's information cannot be changed accidentally or maliciously.

Acceptance Criteria

- The selected club must match an active club-role assignment for the user.
- A Club Admin can manage only the assigned club.
- A Club Representative receives only the permissions configured for that role.
- Requests targeting an unrelated club are rejected.
- Club actions are recorded with the acting user and Club ID.

4.4 Profile Management

4.4.1 US_6.1: Present My Collaboration Strengths

Functional Requirement ID: FR_6 **User Story ID:** US_6.1 **Priority:** High **Status:** Done

Functional Requirement

The system shall allow a Student to create a profile containing academic information, skills, interests, and availability.

User Story

As a Student, I want to highlight my academic background and skills so that suitable project creators can understand what I could contribute.

Acceptance Criteria

- Name, department, and study year are required.
- A valid Student profile is saved and displayed.
- Multiple skills can be added.
- Multiple interests can be added.
- Skills and interests appear on the profile.
- Invalid or duplicate skill entries are handled according to the configured validation rules.
- Other authorized users can view the permitted academic and skill information.

4.4.2 US_6.2: Set My Availability Preferences

Functional Requirement ID: FR_6 **User Story ID:** US_6.2 **Priority:** High **Status:** Done

Functional Requirement

The system shall allow a Student to create a profile containing academic information, skills, interests, and availability.

User Story

As a Student, I want to state whether I am currently open to projects or mentorship so that I receive relevant approaches instead of unsuitable requests.

Acceptance Criteria

- The Student can set project-availability status.
- The Student can set mentorship-availability status.
- Availability changes are saved immediately.
- Availability is shown in relevant teammate or mentor searches.
- Users marked unavailable are excluded when an availability-only filter is applied.

4.4.3 US_7.1: Present My Academic Background

Functional Requirement ID: FR_7 **User Story ID:** US_7.1 **Priority:** High **Status:** Done

Functional Requirement

The system shall allow an Alumni member to create a profile containing graduation and professional information.

User Story

As an Alumni member, I want to present my graduation and career background so that students can evaluate the relevance of my opportunities and guidance.

Acceptance Criteria

- Graduation year and major are required.
- The completed Alumni profile is saved and displayed.
- Company, position, and industry can be added.
- LinkedIn URL is optional.
- Invalid URLs are rejected.
- Saved profile is displayed.

4.4.4 US_7.2: Present My Professional Background

Functional Requirement ID: FR_7 **User Story ID:** US_7.2 **Priority:** High **Status:** Done

Functional Requirement

The system shall allow an Alumni member to create a profile containing graduation and professional information.

User Story

As an Alumni member working in a particular company or industry, I want that information to be discoverable so that students seeking relevant career insight can find me.

Acceptance Criteria

- Alumni can be searched by company.
- Alumni can be filtered by industry.
- Alumni can be filtered by graduation year or programme.
- Search results display a relevant profile summary.
- Only profile information permitted by privacy rules is searchable.

4.4.5 US_8.1: Keep My Personal Profile Updated

Functional Requirement ID: FR_8 **User Story ID:** US_8.1 **Priority:** Medium **Status:** Done

Functional Requirement

The system shall allow an authenticated user to update the editable information in their personal role-specific profile.

User Story

As a member, I want to update my profile, so that my information stays correct and up-to-date.

Acceptance Criteria

- The edit form shows the user's current personal profile values.
- Only fields relevant to the user's role are shown.
- Required personal fields cannot be empty.
- Valid changes are saved.
- Club organizational information is not edited through the personal-profile form.

4.4.6 US_9.1: Evaluate a potential connection

Functional Requirement ID: FR_9 **User Story ID:** US_9.1 **Priority:** Medium **Status:** In Progress

Functional Requirement

The system shall allow an authenticated user to view the permitted personal-profile information of another user.

User Story

As a member, I want to view another member's profile, so that I can learn about them.

Acceptance Criteria

- Privacy settings are respected.
- The user's name and role are displayed.
- Relevant academic or professional information is shown.
- Connection status is displayed when applicable.
- Shared inserests and skills are displayed.
- Private fields remain hidden.

4.4.7 US_9.2: Evaluate a project candidate

Functional Requirement ID: FR_9 **User Story ID:** US_9.2 **Priority:** Medium **Status:** In Progress

Functional Requirement

The system shall allow an authenticated user to view the permitted personal-profile information of another user.

User Story

As a Project Creator reviewing an applicant, I want to inspect the applicant's skills and academic background so that I can assess their suitability for my team.

Acceptance Criteria

- The applicant's relevant skills are displayed.
- The applicant's department and study year are displayed when permitted.
- The application message is accessible from the project-application workflow.
- Only the creator of the relevant project can access applicant-review details.
- The profile view does not expose unrelated private information.

4.5 Club Administration and Announcements

4.5.1 US_10: Create Club Profile

Functional Requirement ID: FR_10 **User Story ID:** US_10 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow a System Admin to create a profile for a university club.

User Story

As a System Admin, I want to create a university club profile so that the club can operate as an independent organization on UniConnect.

Acceptance Criteria

- Only a System Admin can create a club profile.
- Club name, description, and category are required.
- Each club receives a unique Club ID.
- Duplicate club names are rejected.
- The club is created independently of any individual
- user account.
- A successfully created club receives an Active or
- Pending status.

4.5.2 US_11: View Club Profile

Functional Requirement ID: FR_11 **User Story ID:** US_11 **Priority:** Medium **Status:** To Do

Functional Requirement

The system shall allow an authenticated user to view the profile of a university club.

User Story

As an Authenticated User, I want to view a club profile so that I can learn about the club and its activities.

Acceptance Criteria

- The club name, description, category, and logo are
- displayed.
- Club contact information is displayed when available.
- Only active clubs are visible to general users.

- The club profile displays its published
- announcements.
- Selecting a club from the club directory opens its
- profile.

4.5.3 US_12: Assign Initial Club Admin

Functional Requirement ID: FR_12 **User Story ID:** US_12 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow a System Admin to assign a verified university user as the initial administrator of a specific club.

User Story

As a System Admin, I want to assign a verified user as the initial Club Admin so that the selected club can be managed by an authorized person.

Acceptance Criteria

- The selected user must have a verified account.
- The selected club must exist.
- The selected club must be Active or approved for
- operation.
- The assignment identifies one specific club.
- Duplicate active assignments for the same user and
- club are rejected.
- The assigned user retains normal Student access.
- The assigned user receives access to the selected club
- dashboard.
- The assignment is recorded with the assigning System
- Admin and timestamp.

4.5.4 US_13: Add Club Representative

Functional Requirement ID: FR_13 **User Story ID:** US_13 **Priority:** Medium **Status:** To Do

Functional Requirement

The system shall allow a Club Admin to assign a verified university user as a representative of their assigned club.

User Story

As a Club Admin, I want to add a verified user as a Club Representative so that they can help manage club announcements.

Acceptance Criteria

- The requesting user must be an active Club Admin of
- the selected club.
- The selected user must have a verified UniConnect
- account.
- The selected user must not already hold the same
- active role in that club.
- The assignment is linked to one specific club.
- The representative receives permission only for the
- assigned club.
- The representative cannot manage unrelated clubs or
- assign Club Admins.
- The representative receives an assignment
- notification.

4.5.5 US_14: View Club Operators

Functional Requirement ID: FR_14 **User Story ID:** US_14 **Priority:** Medium **Status:** To Do

Functional Requirement

The system shall allow a Club Admin to view the authorized users assigned to their club.

User Story

As a Club Admin, I want to view the users authorized to operate my club so that I can monitor who has club- management access.

Acceptance Criteria

- Only an authorized Club Admin can view the operator
- list.
- The list displays each operator's name and assigned

- club role.
- The assignment status is displayed.
- The assignment date is displayed.
- Operators of unrelated clubs do not appear.
- Inactive assignments are clearly identified or
- excluded.

4.5.6 US_15: Remove Club Representative

Functional Requirement ID: FR_15 **User Story ID:** US_15 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow a Club Admin to revoke a Club Representative's access to their assigned club.

User Story

As a Club Admin, I want to remove a Club Representative so that former representatives can no longer manage club announcements.

Acceptance Criteria

- The requesting user must be an active Club Admin of
- the selected club.
- Only active Club Representative assignments can be removed.
- Removal requires confirmation.
- The removed representative immediately loses club-
- management access.
- The representative's normal Student access remains
- unchanged.
- Other club operators remain unaffected.
- The removal is recorded with date and responsible
- Club Admin.

4.5.7 US_16: Remove Club Admin

Functional Requirement ID: FR_16 **User Story ID:** US_16 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow a System Admin to revoke a user's Club Admin assignment.

User Story

As a System Admin, I want to remove a Club Admin assignment so that an unauthorized or former committee member can no longer manage the club.

Acceptance Criteria

- Only a System Admin can remove a Club Admin
- assignment.
- The selected assignment must be active.
- Removal requires confirmation.
- The club profile and announcements remain available.
- The removed user loses access to the club dashboard.
- The removed user's normal Student account remains
- active.
- The removal is recorded in an audit log.

4.5.8 US_17: Detect Available Dashboards

Functional Requirement ID: FR_17 **User Story ID:** US_17 **Priority:** High **Status:** To Do

Functional Requirement

The system shall determine the available dashboard contexts for an authenticated user after login.

User Story

As an Authenticated User, I want the system to identify my available dashboards so that I can access the roles assigned to me.

Acceptance Criteria

- The system identifies whether the user has Student
- access.
- The system retrieves all active club-role assignments for the user.
- Inactive, expired, or removed assignments are

- excluded.
- Each club context includes the club name and assigned
- role.
- A user with no club assignment is directed to the
- Student dashboard.
- The context list is generated only after successful
- authentication.

4.5.9 US_18: Select Dashboard Context

Functional Requirement ID: FR_18 **User Story ID:** US_18 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow a user with multiple access contexts to select the dashboard they want to use.

User Story

As a User with Student and Club responsibilities, I want to choose my active dashboard so that I can access the functions relevant to my current task.

Acceptance Criteria

- The selection screen appears when more than one
- context is available.
- The Student dashboard option is shown for an eligible
- Student.
- Each active club assignment appears as a separate
- option.
- Selecting Student opens the Student dashboard.
- Selecting a club opens that specific club dashboard.
- The selected context does not grant access to unrelated
- clubs.
- The user does not need to enter the password again.

4.5.10 US_19: Switch Dashboard Context

Functional Requirement ID: FR_19 **User Story ID:** US_19 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow a user to switch between their available Student and Club dashboard contexts without logging in again.

User Story

As a User with multiple responsibilities, I want to switch dashboards without logging in again so that I can move efficiently between Student and Club activities.

Acceptance Criteria

- A context-switching option is available after login.
- Only contexts currently assigned to the user can be
- selected.
- Switching to Student context displays Student
- functions.
- Switching to Club context displays functions for the
- selected club only.
- The authentication session remains active during the
- switch.
- Revoked club assignments disappear from the
- available contexts.

4.5.11 US_20: Create Club Announcement

Functional Requirement ID: FR_20 **User Story ID:** US_20 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow an authorized Club Admin or Club Representative to create a priority-based announcement for their assigned club.

User Story

As an Authorized Club Operator, I want to create a club announcement so that Students can receive club updates.

Acceptance Criteria

- The user has an active Club Admin or Club Representative assignment.

- The assignment belongs to the selected club.
- Announcement title and content are required.
- The announcement is linked to exactly one club.
- The system stores the creator and publication time.
- Unauthorized users cannot publish for the club.
- A successful announcement appears in the club
- announcement feed.
- Priority can be Normal, Important, or Urgent.
- Important or Urgent announcements trigger the configured notification behaviour.
- The announcement displays its publishing club.

4.5.12 US_21: Edit Club Announcement

Functional Requirement ID: FR_21 **User Story ID:** US_21 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow an authorized Club Operator to update a club announcement they are permitted to manage.

User Story

As an Authorized Club Operator, I want to edit a club announcement so that incorrect or outdated informationcan be corrected.

Acceptance Criteria

- The user has an active role in the announcement's club.
- A Club Representative may edit only announcements they created unless broader permission is granted.
- A Club Admin may edit announcements belonging to
- their assigned club.
- Required fields cannot be empty.
- The system records the last modification time.
- Unauthorized update attempts are rejected.
- Updated content appears in the Student announcement feed.

4.5.13 US_22: Delete Club Announcement

Functional Requirement ID: FR_22 **User Story ID:** US_22 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow an authorized Club Operator to delete a club announcement they are permitted to manage.

User Story

As an Authorized Club Operator, I want to delete an obsolete club announcement so that Students do not see outdated information.

Acceptance Criteria

- The user has an active role in the selected club.
- A Club Representative may delete only announcements they created unless broader permission is granted.
- A Club Admin may delete announcements belonging to their assigned club.
- Deletion requires confirmation.
- Deleted announcements no longer appear in the
- Student feed.
- The deletion action is recorded in an audit log.

4.5.14 US_23: View Club Announcements

Functional Requirement ID: FR_23 **User Story ID:** US_23 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow a Student to view announcements published by active university clubs.

User Story

As a Student, I want to view club announcements so that I can stay informed about university club activities.

Acceptance Criteria

- Published announcements from active clubs are displayed.
- Each announcement displays title, club name, content, and publication time.
- Announcements are ordered from newest to oldest.
- Deleted announcements are not displayed.

- Students can open an announcement to view full content.
- Students do not need to join a club to view public announcements.

4.5.15 US_24: Filter Club Announcements

Functional Requirement ID: FR_24 **User Story ID:** US_24 **Priority:** Low **Status:** To Do

Functional Requirement

The system shall allow a Student to filter club announcements by club, category, priority or publication date.

User Story

As a Student, I want to filter club announcements so that I can quickly find updates relevant to me.

Acceptance Criteria

- Announcements can be filtered by club.
- Announcements can be filtered by club category.
- Announcements can be filtered by publication date or date range.
- Announcements can be filtered by priority.
- Multiple filters can be applied together.
- Clearing filters restores the complete announcement
- feed.
- Results contain only announcements matching the
- selected filters.

4.6 Connection and Networking

4.6.1 US_25: Build a relevant network

Functional Requirement ID: FR_25 **User Story ID:** US_25 **Priority:** High **Status:** In Progress

Functional Requirement

The system shall allow an authenticated member to send a connection request to another eligible member.

User Story

As a member, I want to send connection requests to people with similar academic or professional interests, so that I can communicate with them within the platform.

Acceptance Criteria

- Request can be sent from an eligible profile.
- Self-requests are blocked.
- Duplicate pending requests are blocked.
- Request is stored as Pending.
- Recipient receives a notification.

4.6.2 US_26: Control my network

Functional Requirement ID: FR_26 **User Story ID:** US_26 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow the recipient of a pending connection request to accept or reject it.

User Story

As a member, I want to review and accept or decline connections so that my network contains people I genuinely want to engage with.

Acceptance Criteria

- Pending requests are listed.
- Accept and Reject actions are available.
- Accept creates a mutual connection.
- Reject closes the request.
- Requester is notified of acceptance.

4.6.3 US_27: Access My Existing Connections

Functional Requirement ID: FR_27 **User Story ID:** US_27 **Priority:** Medium **Status:** To Do

Functional Requirement

The system shall allow an authenticated member to view their list of first-degree connections.

User Story

As a member, I want to view my list of first-degree connections, so that I can easily see and access my existing professional network.

Acceptance Criteria

- All first-degree connections are displayed.
- Total connection count is shown.
- Connections can be searched by name.
- Connections can be sorted.
- Selecting a connection opens the profile.

4.6.4 US_28: Remove an Outdated Connection

Functional Requirement ID: FR_28 **User Story ID:** US_28 **Priority:** Medium **Status:** To Do

Functional Requirement

The system shall allow an authenticated member to remove an existing first-degree connection.

User Story

As a connected member, I want to remove a connection that is no longer relevant so that future access and communication reflect my current network.

Acceptance Criteria

- Remove option is shown for connections.
- Confirmation is required.
- Relationship is removed for both members.
- Counts are updated.
- Connection-only chat is disabled.

4.7 Project Teammate Finder

4.7.1 US_29.1: Find Teammate Using Skill

Functional Requirement ID: FR_29 **User Story ID:** US_29.1 **Priority:** High **Status:** In Progress

Functional Requirement

The system shall allow a Student to search for potential teammates using required skills.

User Story

As a Student, I want to search for teammates with required skills, so that I can easily find people who have the expertise I need for my projects.

Acceptance Criteria

- At least one required skill is entered.
- Search returns Students whose profiles contain the required skills.
- Department, study year, and availability filters are available.
- Results show the candidate's relevant skills.
- Candidates who do not meet required-skill conditions are excluded or ranked lower according to the matching rule.
- Selecting a result opens the candidate's permitted profile.

4.7.2 US_29.2: Explore Collaborators Before Choosing a Project

Functional Requirement ID: FR_29 **User Story ID:** US_29.2 **Priority:** High **Status:** In Progress

Functional Requirement

The system shall allow a Student to search for potential teammates using required skills.

User Story

As a Student who has not finalized a project idea, I want to discover people with complementary skills so that we can discuss possible projects before building a new project.

Acceptance Criteria

- The search can be performed without selecting or creating a project.
- Multiple required or optional skills can be used for exploratory discovery.
- Results can include candidates outside the searcher's existing connections.
- The search does not create a project or application record.

4.7.3 US_30: Discover Compatible Teammates

Functional Requirement ID: FR_30 **User Story ID:** US_30 **Priority:** High **Status:** In Progress

Functional Requirement

The system shall recommend potential teammates according to skill compatibility, connection proximity, interests.

User Story

As a Student, I want to receive recommendations for potential teammates based on skills, connection proximity, and interests, so that I can discover well-matched collaborators.

Acceptance Criteria

- Each candidate receives a 0-100% score.
- Required skills affect the score.
- Connection proximity affects the score.
- Interests affect the score.
- Results are ordered highest to lowest.

4.7.4 US_31: Discover Suitable Project Opportunities

Functional Requirement ID: FR_31 **User Story ID:** US_31 **Priority:** High **Status:** In Progress

Functional Requirement

The system shall allow a Student to browse available project posts using filters and sorting options.

User Story

As a Student, I want to browse available project posts using filters and sorting options, so that I can find projects that match my skills and schedules efficiently.

Acceptance Criteria

- Open projects are displayed.
- Skill, department, and status filters are available.
- Sorting by date or deadline is available.
- Project cards show key details.
- Results are paginated.

4.7.5 US_32: Evaluate a Project Before Applying

Functional Requirement ID: FR_32 **User Story ID:** US_32 **Priority:** High **Status:** In Progress

Functional Requirement

The system shall allow a Student to view the complete information of a selected project post.

User Story

As a Student, I want to view the complete information of a selected project post, so that I can fully understand the project scope and requirements before deciding to join.

Acceptance Criteria

- Title and description are shown.

- Required skills are shown.
- Creator is shown.
- Remaining positions are shown.
- Current application status is shown.

4.7.6 US_33: Recruit Suitable Project Teammates

Functional Requirement ID: FR_33 **User Story ID:** US_33 **Priority:** High **Status:** In Progress

Functional Requirement

The system shall allow a Student to create a project post containing project information, required skills, and available team positions.

User Story

As a Student, I want to create a project post containing project details and required skills, so that I can attract suitable teammates who can contribute to my project.

Acceptance Criteria

- Title and description are required.
- At least one skill is required.
- Team positions must be greater than one.
- Post is created as Open.
- Creator is added as a member.

4.7.7 US_34: Apply to Join a Project

Functional Requirement ID: FR_34 **User Story ID:** US_34 **Priority:** High **Status:** In Progress

Functional Requirement

The system shall allow an eligible Student to submit an application to an open project.

User Story

As an eligible Student, I want to apply to an open project, so that I can join projects that interest me and fit my skills.

Acceptance Criteria

- Only Open projects accept applications.
- Duplicate applications are blocked.
- Optional message is limited to 500 characters.

- Application is stored as Pending.

4.7.8 US_35: Review Project Applications

Functional Requirement ID: FR_35 **User Story ID:** US_35 **Priority:** High **Status:** In Progress

Functional Requirement

The system shall allow the creator of a project to view applications submitted to that project.

User Story

As the creator of a project, I want to view applications submitted to that project, so that I can review interested students and evaluate their suitability.

Acceptance Criteria

- Only the creator can view applications.
- Application status is visible.
- Applicant profile summary is shown.
- Applicant message is shown.
- Applications can be ordered by time.

4.7.9 US_36: Select Project Team Members

Functional Requirement ID: FR_36 **User Story ID:** US_36 **Priority:** High **Status:** In Progress

Functional Requirement

The system shall allow the creator of a project to accept or reject a pending project application.

User Story

As the creator of a project, I want to accept suitable applicants and decline unsuitable ones, so that I can select the best candidates for my team.

Acceptance Criteria

- Only Pending applications can be processed.
- Accept changes status to Accepted.
- Accepted applicant is added to the team.
- Reject changes status to Rejected.
- Applicant receives the decision.

4.7.10 US_37: Update Project Requirements

Functional Requirement ID: FR_37 **User Story ID:** US_37 **Priority:** Medium **Status:** To Do

Functional Requirement

The system shall allow the creator of a project to update its editable information.

User Story

As the creator of a project, I want to update the project post, so that I can keep project details accurate and relevant for the future applicants.

Acceptance Criteria

- Only creator can edit.
- Current values are shown.
- Required fields cannot be removed.
- Invalid values are rejected.
- Valid changes appear immediately.

4.7.11 US_38.1: Close Project Recruitment

Functional Requirement ID: FR_38 **User Story ID:** US_38.1 **Priority:** Medium **Status:** In Progress

Functional Requirement

The system shall allow the creator of a project to close or reopen the project.

User Story

As a Project Creator whose available positions are filled, I want to close recruitment so that students do not spend time applying to an unavailable project.

Acceptance Criteria

- Only creator can change status.
- Closing changes the project status to Closed.
- Closed projects reject new applications.
- Existing applications remain available for review according to policy.
- The project remains visible unless separately deleted.
- Potential applicants can see that recruitment is closed..
- Updated status is displayed.

4.7.12 US_38.2: Reopen Project Recruitment

Functional Requirement ID: FR_38 **User Story ID:** US_38.2 **Priority:** Medium **Status:** In Progress

Functional Requirement

The system shall allow the creator of a project to close or reopen the project.

User Story

As a Project Creator whose team has gained another vacancy, I want to reopen recruitment so that new candidates can apply.

Acceptance Criteria

- Reopening is allowed only when an open position is available.
- Reopening changes the project status to Open.
- New applications are accepted after reopening.
- The project returns to open-project browse results.
- The reopened status is visible immediately.

4.7.13 US_39: Remove an Abandoned Project

Functional Requirement ID: FR_39 **User Story ID:** US_39 **Priority:** High **Status:** In Progress

Functional Requirement

The system shall allow the creator of a project to permanently delete the project after confirmation.

User Story

As the creator of a project, I want to delete the project after confirmation, so that I can remove outdated or abandoned projects from the system.

Acceptance Criteria

- Only creator can delete.
- Confirmation is required.
- Cancel keeps the project unchanged.
- Confirm removes it from browse and search.
- Related applicants are notified when needed.

4.8 Peer Mentorship

4.8.1 US_40: Offer Peer Academic Mentorship

Functional Requirement ID: FR_40 **User Story ID:** US_40 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow an eligible senior Student to register as a peer mentor by providing expertise and mentoring information.

User Story

As an eligible senior Student, I want to register as a peer mentor by providing my expertise on comlpleted courses of projects, so that I can help junior students with university-specific guidance.

Acceptance Criteria

- Minimum study-year rule is checked.
- At least one expertise area is required.
- Mentoring bio is required.
- Duplicate registration is blocked.
- Mentor appears in the directory.

4.8.2 US_41: Offer Alumni Career Mentorship

Functional Requirement ID: FR_41 **User Story ID:** US_41 **Priority:** Low **Status:** To Do

Functional Requirement

The system shall allow an Alumni to register as a mentor by providing expertise and mentoring information.

User Story

As an Alumni, I want to register as a mentor by providing my expertise and mentoring information, so that I can share my professional experience and guide students in their career development.

Acceptance Criteria

- The applicant must be logged in with a verified Alumni account.
- The Alumni can provide at least one area of expertise.
- The Alumni must provide a mentoring bio or description of the guidance they can offer.

- The system must reject duplicate mentor registration for the same Alumni account.
- Successful registration must create an Alumni Mentor profile with an initial status of Available.
- The registered Alumni Mentor must appear in the mentor directory.
- The system must allow the registered Alumni Mentor to create mentorship slots after successful registration.
- If required information is missing or invalid, the system must display a clear validation message.

4.8.3 US_42: Publish a Mentorship Slot

Functional Requirement ID: FR_42 **User Story ID:** US_42 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow a registered Mentor to create a scheduled mentorship slot.

User Story

As a registered Mentor, I want to create a scheduled mentorship slot, so that students can book time with me for guidance and support.

Acceptance Criteria

- Topic, description, date, and times are required.
- End time must be later than start time.
- Capacity must be within the allowed range.
- Conflicting slots are rejected.
- Valid slot is created as Open.

4.8.4 US_43: Find a Mentor with Relevant Expertise

Functional Requirement ID: FR_43 **User Story ID:** US_43 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow a Student to search and browse available mentors using expertise-related filters.

User Story

As a Student, I want to search and browse available mentors using expertise-related filters, so that I can find mentors who specialize in areas relevant to my needs.

Acceptance Criteria

- Available mentors are displayed.
- Expertise filter is available.
- Department and year filters are available.
- Profile summary is shown.
- Selecting a mentor shows available slots.

4.8.5 US_44: Find a Suitable Mentorship Session

Functional Requirement ID: FR_44 **User Story ID:** US_44 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow a Student to browse available mentorship slots using topic, schedule, mode, and availability filters.

User Story

As a Student, I want to browse available mentorship slots using topic, schedule, mode, and availability filters, so that I can find sessions that fit my learning goals and availability.

Acceptance Criteria

- Open slots are displayed.
- Topic, day, time, and location filters are available.
- Enrollment and capacity are shown.
- Full and Closed slots are labelled.
- Selecting a slot opens its details.

4.8.6 US_45: Reserve a Mentorship Place

Functional Requirement ID: FR_45 **User Story ID:** US_45 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow a Student to join an open mentorship slot with available capacity.

User Story

As a Student, I want to join an open mentorship slot with available capacity, so that I can receive mentorship from experienced individuals.

Acceptance Criteria

- Slot must be Open.
- Capacity must be available.

- Duplicate joining is blocked.
- Enrollment count increases by one.
- Mentor and mentee receive confirmation.

4.8.7 US_46: Release My Mentorship Place

Functional Requirement ID: FR_46 **User Story ID:** US_46 **Priority:** Medium **Status:** To Do

Functional Requirement

The system shall allow an enrolled Student to leave a mentorship slot.

User Story

As an enrolled Student, I want to leave a mentorship slot, so that I can free up my schedule and allow others to take the spot if my circumstances change.

Acceptance Criteria

- Only enrolled mentees can leave.
- Confirmation is required.
- Student is removed from the list.
- Enrollment count decreases.
- Full slot becomes Open when space is available.

4.8.8 US_47: Update Mentorship Session Details

Functional Requirement ID: FR_47 **User Story ID:** US_47 **Priority:** Medium **Status:** To Do

Functional Requirement

The system shall allow the creator of a mentorship slot to update its permitted details.

User Story

As a Mentorship Slot Creator, I want to update the permitted details of my mentorship slot, so that I can keep the mentorship information accurate and up-to-date.

Acceptance Criteria

- Only creator can edit.
- New schedule is checked for conflicts.
- Capacity cannot be below current enrollment.
- Invalid values are rejected.
- Mentees are notified of important changes.

4.8.9 US_48: Close Mentorship Enrollment

Functional Requirement ID: FR_48 **User Story ID:** US_48 **Priority:** Medium **Status:** To Do

Functional Requirement

The system shall allow the creator of a mentorship slot to stop accepting new mentees.

User Story

As the creator of a mentorship slot, I want to stop accepting new mentees, so that I can manage my capacity and focus on existing mentees.

Acceptance Criteria

- Only creator can close.
- Status becomes Closed.
- Existing mentees remain enrolled.
- New joins are rejected.
- Closed status is visible.

4.8.10 US_49: Cancel a Mentorship Session

Functional Requirement ID: FR_49 **User Story ID:** US_49 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow the creator of a mentorship slot to cancel the slot and notify its enrolled mentees.

User Story

As the creator of a mentorship slot, I want to cancel the slot when I can no longer conduct a session and notify its enrolled mentees, so that they are informed in advance and can make alternative arrangements.

Acceptance Criteria

- Only creator can cancel.
- Confirmation is required.
- Status becomes Cancelled.
- New enrollment is blocked.
- All enrolled mentees are notified.

4.8.11 US_50: Track My Joined Mentorship Sessions

Functional Requirement ID: FR_50 **User Story ID:** US_50 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow a Student to view the mentorship slots they created or joined.

User Story

As a Student, I want to view the mentorship slots I created or joined, so that I can track my mentoring activities and commitments.

Acceptance Criteria

- Mentors see created slots.
- Mentees see joined slots.
- Next date and time are shown.
- Location or meeting link is shown.
- Slot status is shown.

4.9 One-to-One Chat

4.9.1 US_51: Continue a Discussion in Real Time

Functional Requirement ID: FR_51 **User Story ID:** US_51 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow an registered member to send a real-time text message to another registered member.

User Story

As a registered member, I want to send a real-time text message to another registered member, so that I can communicate instantly and collaborate effectively with others.

Acceptance Criteria

- Relationship rule is checked.
- Empty messages are rejected.
- Message length is limited to 5,000 characters.
- Message is stored with timestamp.
- Online recipient receives it without refresh.

4.9.2 US_52: Receive New Messages Instantly

Functional Requirement ID: FR_52 **User Story ID:** US_52 **Priority:** High **Status:** To Do

Functional Requirement

The system shall deliver incoming real-time messages to the eligible recipient.

User Story

As a member, I want to receive real-time messages, so that I can communicate quickly.

Acceptance Criteria

- Active chat receives incoming messages.
- Sender identity is shown.
- Timestamp is shown.
- Unread count updates when inactive.

4.9.3 US_53: Resume Recent Conversations

Functional Requirement ID: FR_53 **User Story ID:** US_53 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow an authenticated member to view their conversations ordered by most recent activity.

User Story

As a member, I want to view my conversations, so that I can access recent chats.

Acceptance Criteria

- All conversations are shown.
- Most recent conversation appears first.
- Participant name is shown.
- Latest message preview is shown.
- Unread count is shown.

4.9.4 US_54: Review Previous Conversation Context

Functional Requirement ID: FR_54 **User Story ID:** US_54 **Priority:** Medium **Status:** To Do

Functional Requirement

The system shall allow an authenticated member to view the stored message history of a selected conversation.

User Story

As a member, I want to view message history, so that I can read previous conversations.

Acceptance Criteria

- Messages are chronological.
- Sender and timestamp are shown.
- Older messages can be loaded.
- Opening marks unread messages as read.
- New messages can be sent from the same view.

4.9.5 US_55: Prevent Unwanted Direct Messages

Functional Requirement ID: FR_55 **User Story ID:** US_55 **Priority:** High **Status:** To Do

Functional Requirement

The system shall prevent one-to-one messaging between members who do not satisfy the required relationship conditions.

User Story

As a member, I want direct messaging limited to permitted relationships so that strangers cannot contact me without first establishing an accepted connection or other approved relationship.

Acceptance Criteria

- Relationship is checked before chat opens.
- Ineligible users cannot send messages.
- Connect-to-message instruction is shown.
- Removing the relationship disables new messages.

4.10 Career Board

4.10.1 US_56: Publish a Career Opportunity

Functional Requirement ID: FR_56 **User Story ID:** US_56 **Priority:** High **Status:** Done

Functional Requirement

The system shall allow a verified Alumni member to create a job or internship post.

User Story

As a Verified Alumni Member, I want to create a job or internship post, so that I can share career opportunities with students.

Acceptance Criteria

- Only verified Alumni can post.
- Position, company, type, and description are required.
- Application URL or email is required.
- Post appears on the career board.

4.10.2 US_57: Find Relevant Career Opportunities

Functional Requirement ID: FR_57 **User Story ID:** US_57 **Priority:** High **Status:** Done

Functional Requirement

The system shall allow a Student to browse job and internship posts using relevant filters and sorting options.

User Story

As a Student, I want to browse job and internship posts using relevant filters and sorting options, so that I can find career opportunities that match my interests and qualifications.

Acceptance Criteria

- Active posts are displayed.
- Type, location, and skill filters are available.
- Sorting by date or deadline is available.
- Job cards show key details.
- Expired posts are marked or removed.

4.10.3 US_58: Evaluate a Career Opportunity

Functional Requirement ID: FR_58 **User Story ID:** US_58 **Priority:** High **Status:** In Progress

Functional Requirement

The system shall allow a Student to view the complete details of a selected job or internship post.

User Story

As a Student, I want to view the complete details of a selected job or internship post, so that I can understand the opportunity and decide whether it is suitable for me.

Acceptance Criteria

- Position and company are shown.
- Description and qualifications are shown.
- Required skills are shown.
- Deadline and application method are shown.

4.10.4 US_59: Build My Application Shortlist

Functional Requirement ID: FR_59 **User Story ID:** US_59 **Priority:** Medium **Status:** In Progress

Functional Requirement

The system shall allow a Student to save a job or internship post for later review.

User Story

As a Student, I want to save a job or internship post for later review, so that I can easily return to opportunities that interest me.

Acceptance Criteria

- Save action is available on active posts.
- Post is added to Saved Jobs.
- Duplicate saves are blocked.
- Saved status is visible.
- Saved post remains until removed or deleted.

4.10.5 US_60: Remove an Opportunity from My Shortlist

Functional Requirement ID: FR_60 **User Story ID:** US_60 **Priority:** Medium **Status:** In Progress

Functional Requirement

The system shall allow a Student to remove a job or internship post from their saved list.

User Story

As a Student, I want to remove a job or internship post from my saved list, so that I can keep my saved list contains my preferred posts.

Acceptance Criteria

- Remove action appears only for saved posts.
- Bookmark record is deleted.
- Original post remains unchanged.
- Post disappears from Saved Jobs.
- It can be saved again later.

4.10.6 US_61: Continue to the Official Application Channel

Functional Requirement ID: FR_61 **User Story ID:** US_61 **Priority:** High **Status:** In Progress

Functional Requirement

The system shall redirect a Student to the external application link or email provided in a career post.

User Story

As a Student, I want to access the external application link or email provided in a career post, so that I can apply for the job or internship through the specified application method.

Acceptance Criteria

- Apply action appears when details exist.
- Valid URL opens the application page.
- Application email opens the email client.

4.11 Global Search

4.11.1 US_62: Explore Results Related to a Topic

Functional Requirement ID: FR_62 **User Story ID:** US_62 **Priority:** Low **Status:** In Progress

Functional Requirement

The system shall allow an authenticated member to search across users, projects, clubs, events, and career opportunities.

User Story

As an Authenticated Member, I want to search across users, projects, clubs, events, and career opportunities, so that I can quickly find relevant information and opportunities on the platform.

Acceptance Criteria

- The query contains at least two characters.
- All supported entity types are searchable.
- Club results come from Club records rather than Club Admin profiles.
- Club results display club name, category, logo, and description.
- Results are grouped by entity type and ranked by relevance.
- Inactive or suspended clubs are excluded from general results.
- Selecting a club result opens the Club profile.

4.12 Administration and Moderation

4.12.1 US_63: Locate an Account Requiring Action

Functional Requirement ID: FR_63 **User Story ID:** US_63 **Priority:** Low **Status:** To Do

Functional Requirement

The system shall allow a System Admin to search registered user accounts.

User Story

As a System Admin, I want to search registered user accounts, so that I can quickly find and manage specific users when needed.

Acceptance Criteria

- Search by name is available.
- Search by email is available.
- Role and account status are shown.
- Partial terms return matches.
- Selecting a result opens account details.

4.12.2 US_64: Temporarily Restrict a User Account

Functional Requirement ID: FR_64 **User Story ID:** US_64 **Priority:** Low **Status:** To Do

Functional Requirement

The system shall allow a System Admin to suspend a registered user account.

User Story

As a System Admin, I want to suspend a registered user account, so that I can temporarily restrict access for users who violate platform rules.

Acceptance Criteria

- Only System Admin can suspend.
- Confirmation is required.
- Status becomes Suspended.
- Suspended account cannot log in.
- Action is recorded in an audit log.

4.12.3 US_65: Restore a Suspended User's Access

Functional Requirement ID: FR_65 **User Story ID:** US_65 **Priority:** Low **Status:** To Do

Functional Requirement

The system shall allow a System Admin to reactivate a suspended user account.

User Story

As a System Admin, I want to reactivate a suspended user account, so that I can restore access for users after they have resolved the issues.

Acceptance Criteria

- Only suspended accounts can be reactivated.
- Only System Admin can reactivate.
- Status becomes Active.
- Member can log in again.
- Action is recorded in an audit log.

4.12.4 US_66: Remove a Permanently Invalid Account

Functional Requirement ID: FR_66 **User Story ID:** US_66 **Priority:** Low **Status:** To Do

Functional Requirement

The system shall allow a System Admin to delete a user account after confirmation.

User Story

As a System Admin, I want to delete a user account after confirmation, so that ineligible accounts are permanently removed.

Acceptance Criteria

- Only System Admin can initiate deletion.
- Explicit confirmation is required.

- Cancel leaves account unchanged.
- Confirm prevents future access.
- Deletion action is recorded.

4.12.5 US_67: Review Reported Content

Functional Requirement ID: FR_67 **User Story ID:** US_67 **Priority:** Low **Status:** To Do

Functional Requirement

The system shall allow a System Admin to view content submitted for moderation.

User Story

As a System Admin, I want to view reported content, so that I can review the flagged contents before taking action.

Acceptance Criteria

- Reports appear in a moderation queue.
- Original content is displayed.
- Report reason is displayed.
- Content owner is identified.
- Reports can be filtered by status.

4.12.6 US_68: Remove Content That Violates Platform Rules

Functional Requirement ID: FR_68 **User Story ID:** US_68 **Priority:** Low **Status:** To Do

Functional Requirement

The system shall allow a System Admin to remove content that violates platform rules.

User Story

As a System Admin, I want to remove offending contents, so that I can maintain a safe and professional environment for all users.

Acceptance Criteria

- Only System Admin can remove moderated content.
- Moderation reason is required.
- Removed content is no longer public.
- Report status becomes Resolved.
- Action is recorded with date and admin.

4.12.7 US_69: Warn a Member About a Violation

Functional Requirement ID: FR_69 **User Story ID:** US_69 **Priority:** Low **Status:** To Do

Functional Requirement

The system shall allow a System Admin to issue a warning to a member responsible for inappropriate content.

User Story

As a System Admin, I want to issue a documented warning to a member responsible for inappropriate content so that the member understands the rule and has an opportunity to correct their behaviour.

Acceptance Criteria

- Warning reason is required.
- Warning is linked to the account.
- Member receives a notification.
- Warning date is recorded.
- Previous warnings remain visible to admins.

4.12.8 US_70: Suspend a Club's Publishing Access

Functional Requirement ID: FR_70 **User Story ID:** US_70 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow a System Admin to suspend a club account.

User Story

As a System Admin investigating a club that has violated platform rules, I want to suspend its publishing access so that no new official content appears while the issue is reviewed.

Acceptance Criteria

- Only System Admins can suspend club.
- Every state change is recorded.
- Suspending a club blocks new publication.

4.12.9 US_71: Remove an Invalid Club Profile

Functional Requirement ID: FR_71 **User Story ID:** US_71 **Priority:** High **Status:** To Do

Functional Requirement

The system shall allow a System Admin to delete a club account.

User Story

As a System Admin, I want to remove a duplicate, dissolved, or invalid club profile after confirmation so that obsolete organizations are removed from the platform.

Acceptance Criteria

- Only System Admins can delete a club.
- Every state change is recorded.
- Deleting a club requires explicit confirmation.

4.13 Notification Management

4.13.1 US_72.1: Receive Notifications Requiring Attention

Functional Requirement ID: FR_72 **User Story ID:** US_72.1 **Priority:** Low **Status:** To Do

Functional Requirement

The system shall generate an in-app notification when a relevant platform or club activity affects a member.

User Story

As a registered member, I want notifications for activities that require my attention, such as connection requests, project decisions, and mentorship changes, so that I can respond promptly.

Acceptance Criteria

- Unread notifications are visually distinguished.
- Notifications are associated with the affected authenticated user.
- A notification is generated for a new connection request.
- A notification is generated for a project-application decision.
- A notification is generated for relevant mentorship enrollment or schedule changes.
- Duplicate notifications for the same event are prevented according to system rules.
- Marking a notification as read updates its state.

4.13.2 US_72.2: Receive Club Access Change Notifications

Functional Requirement ID: FR_72 **User Story ID:** US_72.2 **Priority:** Medium **Status:** In Progress

Functional Requirement

The system shall generate an in-app notification when a relevant platform or club activity affects a member.

User Story

As a user assigned to operate a club, I want to be notified when that role is granted or revoked so that I understand which Club dashboards and responsibilities are available to me.

Acceptance Criteria

- A notification is generated when Club Admin access is granted.
- A notification is generated when Club Representative access is granted.
- A notification is generated when a club-role assignment is revoked.
- The notification identifies the affected club and role.
- Revoked access disappears from available Club contexts.

4.13.3 US_72.3: Open the Item Related to a Notification

Functional Requirement ID: FR_72 **User Story ID:** US_72.3 **Priority:** Medium **Status:** To Do

Functional Requirement

The system shall generate an in-app notification when a relevant platform or club activity affects a member.

User Story

As a member receiving a notification, I want it to open the associated request, project, mentorship, club, or announcement so that I can act without searching for it manually.

Acceptance Criteria

- Selecting a notification opens the related item when the item still exists.
- The link respects the user's current authorization.
- If access was revoked, the notification does not restore or bypass access.
- If the target item is unavailable, a clear message is shown.
- Opening the target may mark the notification as read according to the configured behavior.

Chapter 5

Requirements Traceability and Design Alignment

5.1 Traceability Summary

Requirement Area	Primary Design Diagram	Core Design Elements
Authentication and Profiles	Authentication/Profile Diagram	User, role profiles, verification and reset tokens, academic reference entities
Club Operations	Club/Announcement Diagram	Club, role assignment, announcement, moderation, notification
Networking	Connection Diagram	Connection lifecycle, statistics, relationship notifications
Project Collaboration	Project Teammate Diagram	Project, application, membership, normalized skills, recommendations
Mentorship	Peer Mentorship Diagram	Mentor profile, slot, enrollment, mentor skills, reminders
Communication	One-to-One Chat Diagram	Conversation, participants, messages, delivery status
Career Opportunities	Career Board Diagram	Job posts, skills, saved jobs, external application data
Administration	Administration/Notification Diagram	Moderation actions, audit logs, suspensions, cross-module notifications

5.2 Design Consistency

The high-level diagram provides the shared domain vocabulary, while the feature diagrams refine the entities and operations needed to satisfy individual user stories. The three-tier architecture assigns controllers to presentation coordination, services to the acceptance-criteria rules, and repositories to persistence. This creates a direct trace from a user story to a controller action, business rule, entity relationship, and stored record.

5.3 Conclusion

UniConnect consolidates academic networking, collaboration, mentorship, messaging, club communication, and career discovery into a university-verified platform. The proposed three-tier MVC architecture separates concerns, and the ordered class diagrams demonstrate how the domain grows from shared identity management into specialized feature modules. The detailed SRS provides measurable acceptance criteria for implementation and testing.